

Bitte lesen Sie die Fragen aufmerksam durch und beantworten Sie nur das Gefragte prägnant.

Verwenden Sie die Programmiersprache C für die Implementierungen und Beispiele.

(*main()*, *includes* und *printf* werden in dieser Prüfung nicht benötigt.)

Aufgabe 1) Verständnisfragen (2 + 2 + 1 + 1 Punkte) $2 + 2 + 1 + 0,5 = 5,5$

- Erklären Sie *call-by-value* und zeigen Sie die Verwendung in einem kleinen C-Beispiel.
- Erklären Sie *call-by-reference* und zeigen Sie die Verwendung in einem kleinen C-Beispiel.
- Was bedeutet das Keyword *volatile*? Zeigen Sie eine sinnvolle Verwendung in C.
- Was versteht man unter *Alignment*? Zeigen Sie wie sich das Alignment auf die Verwendung von *structs* in C auswirkt.

Aufgabe 2) Einfach verkettete Liste (1 + 10 Punkte) $1 + 5 = 6$

- Definieren Sie einen Knoten für eine einfach verkettete Liste von Integerwerten als *Node_t*.
- Schreiben Sie eine Funktion *delete_even* in C, die den Head-Pointer auf eine einfach verkettete Liste erhält und alle geraden Knoten aus der Liste löscht.

Aufgabe 3) Speicherverwaltung (2 + 1 + 5 + 5 Punkte) $2 + 1 + 5 + 5 = 13$

Sie möchten eine eigene Speicherverwaltung mit *malloc* und *free* implementieren. Jeder Block hat einen Header, bestehend aus einem *allocated*-Flag, einer Blockgröße (Speicherung exklusive Header-Größe) von 20 Bit und einer Checksumme von 8 Bit.

- Definieren Sie einen möglichst kompakten Header für die einzelnen Blöcke unter Verwendung von Bitdefinitionen für die Maskierung.
- Definieren Sie einen möglichst kompakten Header für die einzelnen Blöcke unter Verwendung eines *Bit Fields*.
- Implementieren Sie eine C-Funktion *mem_freeBlockSize*, die die Größe des größten freien Speicherblocks unter Verwendung des Headers aus 3.a. und Bitmaskierung zurückgibt.
- Implementieren Sie eine C-Funktion *mem_freeMemSize*, die die gesamte Größe aller freien Speicherblöcke unter Verwendung des Headers aus 3.b. und Bit Fields berechnet und zurückgibt.